

CHOFYAI STUDIO

12 · Pruebas y calidad

Documento	12-testing-and-quality.md
Versión analizada	0.5.1 (commit f840055)
Fecha del análisis	2026-08-28
Fuente Markdown	docs/system-documentation/12-testing-and-quality.md
Generado por	scripts/docs/build-pdf.mjs

Contenido

1. Resultado de la ejecución real
2. Inventario de pruebas
 - 2.1 ``src/utls.test.ts`` — 13 pruebas
 - 2.2 ``src/i18n.test.ts`` — 7 pruebas
 - 2.3 ``src-tauri/src/system.rs`` → ``#[cfg(test)] mod tests`` — 4 pruebas
3. Cómo ejecutar las pruebas
4. Cobertura observable
5. Herramientas de calidad configuradas
6. Integración continua
 - ``.github/workflows/ci.yml``
 - ``.github/workflows/security.yml``
7. Datos y fixtures de prueba
8. Casos límite cubiertos y no cubiertos
9. Propuesta priorizada de pruebas faltantes
10. Criterios de aceptación sugeridos

Estado: completo · Última revisión: 2026-08-27 · Versión analizada: 0.5.1 (commit f840055)

Este documento describe qué se prueba hoy, qué no, con qué herramientas se controla la calidad y qué pruebas faltan. Todas las cifras provienen de ejecuciones reales realizadas durante este análisis, no de estimaciones.

1. Resultado de la ejecución real

Las tres comprobaciones se ejecutaron en un Mac con Apple Silicon el 2026-08-27.

Comprobación	Comando	Resultado
Pruebas del frontend	<code>pnpm test</code>	2 archivos, 20 pruebas, todas correctas , 514 ms
Pruebas del backend	<code>cd src-tauri && cargo test --no-default-features</code>	4 pruebas correctas , 0 fallos, 0 ignoradas
Comprobación de tipos	<code>pnpm exec tsc --noEmit</code>	Sin errores, salida vacía

Salida literal de Vitest:

```
RUN v4.1.10 /Volumes/ORICO/ChofyIA/chofyai-studio

Test Files  2 passed (2)
   Tests   20 passed (20)
  Duration  514ms
```

Salida literal de Cargo:

```
test system::tests::delete_model_rejects_path_traversal ... ok
test system::tests::pid_alive_for_self_is_true ... ok
test system::tests::pid_alive_for_zero_is_false ... ok
test system::tests::read_disk_usage_returns_two_values ... ok

test result: ok. 4 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

No se ejecutó `pnpm build:web` ni el empaquetado de Tauri como parte de este análisis.

2. Inventario de pruebas

2.1 `src/utils.test.ts` — 13 pruebas

Bloque	Pruebas	Qué cubre
<code>fmtBytes</code>	2	Valores nulos y cero devuelven <code>—</code> ; formateo de B, KB, MB y GB con el número correcto de decimales
<code>fmtElapsed</code>	1	Formato <code>M:SS</code> en cuatro casos, incluido el cambio de minuto
<code>parseInstallLine</code>	10	Clonado (español e inglés), porcentaje de <code>Receiving objects</code> , porcentaje de cmake, recorte de valores mayores que 100, enlazado, dependencias Python (uv y pip), descarga de modelo, marcador <code>INSTALL_OK</code> , preservación del estado previo cuando la línea no encaja, y limpieza de códigos ANSI

Es el archivo de pruebas más valioso del repositorio: `parseInstallLine` es puro, tiene muchas ramas y su salida se ve directamente en la interfaz. La prueba de "preserva valores previos" es especialmente buena, porque protege el contrato acumulativo de la función.

2.2 `src/i18n.test.ts` — 7 pruebas

Ejecuta con `// @vitest-environment jsdom` porque `setLang` toca `localStorage` y `document.documentElement.lang`.

Prueba	Qué protege
Idioma por defecto es <code>es</code>	Contrato de <code>DEFAULT_LANG</code>
Cambio entre ES y EN	Que los diccionarios se conmuten en caliente
Sustitución de parámetros <code>{clave}</code>	Interpolación de <code>t()</code>
Fallback ante clave inexistente	Que devuelva la clave cruda y no reviente
Rechazo de idiomas no soportados	Que <code>setLang('xx')</code> no cambie nada
Paridad de diccionarios ES/EN	Que toda clave del diccionario por defecto exista también en inglés
<code>SUPPORTED_LANGS</code> contiene <code>es</code> y <code>en</code>	Contrato de la constante

La prueba de paridad es la más útil: convierte un olvido de traducción en un fallo de CI. Se apoya en un detalle sutil —que `t()` devuelve la clave cruda cuando falta— y por eso detectaría también una clave escrita con una errata.

2.3 src-tauri/src/system.rs → #[cfg(test)] mod tests — 4 pruebas

ChofyAI Studio · 12 · Pruebas y calidad

v0.5.1 (commit f840055) · 2026-08-28

Prueba	Qué hace realmente	Valoración
pid_alive_for_self_is_true	Comprueba que pid_is_alive reconozca el PID del propio proceso de test	Correcta y útil
pid_alive_for_zero_is_false	Pese al nombre, prueba con el PID 999_999_999 , no con el cero	El nombre engaña; la prueba en sí es válida
delete_model_rejects_path_traversal	No llama a delete_tool_model : sólo comprueba que la cadena "." contenga "."	Prueba vacía: no ejercita ninguna lógica de producción y da una falsa sensación de cobertura sobre el control de seguridad más importante del backend
read_disk_usage_returns_two_values	Ejecuta df sobre / y comprueba que total >= free y total > 0	Correcta, aunque depende del sistema anfitrión

Conclusión honesta: de las cuatro pruebas de Rust, tres aportan algo y una es decorativa. La guardia de recorrido de rutas de delete_tool_model está sin probar de verdad.

3. Cómo ejecutar las pruebas

```
pnpm test          # Vitest, una pasada
pnpm test:watch    # Vitest en modo observación
pnpm test:rust     # cargo test dentro de src-tauri
pnpm exec tsc --noEmit # comprobación de tipos
```

Diferencia relevante con CI: el trabajo test-rust de .github/workflows/ci.yml ejecuta cargo test --no-default-features , mientras que el script pnpm test:rust ejecuta cargo test sin ese indicador. En un runner de Ubuntu la bandera evita arrastrar dependencias gráficas de Tauri; en local, ejecutar sin ella compila más cosas y tarda más. Es una divergencia menor pero conviene conocerla.

4. Cobertura observable

No hay herramienta de cobertura configurada, así que no existe un porcentaje que citar. Lo que sí se puede afirmar es qué módulos tienen prueba y cuáles no.

Módulo	Líneas aprox.	Tipo de prueba	Cobertura aparente	Riesgo
ChofyAI Studio · 12 · Pruebas y calidad src/utls.ts	53	Unitaria	Alta: las tres funciones y la mayoría de las ramas	Bajo
src/i18n.ts	295	Unitaria	Alta sobre la API pública	Bajo
src/App.tsx	2766	Ninguna	Nula	Alto
src/types.ts	194	Sólo comprobación de tipos	—	Bajo
src-tauri/src/system.rs	2029	4 pruebas	Muy baja: no se prueban rutas, manifiestos, instalación, procesos ni modelos	Alto
src-tauri/src/models.rs	117	Ninguna	Nula, aunque es sólo estructuras	Bajo
src-tauri/src/lib.rs	58	Ninguna	Nula	Medio: un comando sin registrar no lo detecta nada
scripts/mac/*.sh	~740	Ninguna	Nula	Alto
scripts/win/*.ps1	~470	Ninguna	Nula	Alto, y sin validación de extremo a extremo
scripts/docs/build-pdf.mjs	~400	Ninguna	Nula	Medio
apps/*.yaml	—	Validación de campos en CI	Parcial	Medio
workflows/*.yaml	—	Ninguna	Nula	Medio

Lo más llamativo: los dos archivos que concentran el 90 % de la lógica del sistema (`App.tsx` y `system.rs`) son precisamente los menos probados.

5. Herramientas de calidad configuradas

Herramienta	Configuración	Qué comprueba
Vitest	<code>package.json</code> , entorno jsdom por archivo	Pruebas unitarias del frontend
<code>tsc --noEmit</code>	<code>tsconfig.json</code> con <code>strict: true</code>	Tipos de todo <code>src/</code>
<code>cargo test</code>	<code>#[cfg(test)]</code> en <code>system.rs</code>	Pruebas del backend
markdownlint-cli2	<code>.markdownlint.json</code> , <code>.markdownlint-cli2.jsonc</code> , <code>.markdownlintignore</code>	Formato de todos los <code>.md</code>
Validador de manifiestos	Script Python embebido en <code>ci.yml</code>	Campos obligatorios, categorías y runtimes válidos, <code>run</code> si hay <code>install_script</code>
TruffleHog	<code>security.yml</code> , <code>--only-verified</code>	Secretos verificados en el historial
<code>pnpm audit</code>	<code>security.yml</code> , sólo producción, nivel alto	Vulnerabilidades de npm
<code>cargo audit</code>	<code>security.yml</code>	Vulnerabilidades de crates
CodeQL	<code>security.yml</code>	Análisis estático de seguridad
Dependabot	<code>.github/dependabot.yml</code>	Actualizaciones semanales agrupadas

Herramientas que **no** están configuradas, verificado por ausencia de archivos y de dependencias:

- ESLint y Prettier: no hay `.eslintrc*` ni `.prettierrc*` , ni dependencias asociadas. El código contiene un comentario `// eslint-disable-next-line react-hooks/exhaustive-deps` en `App.tsx` , resto de una configuración que hoy no existe.
- `clippy` y `rustfmt` no se ejecutan en CI.
- No hay medición de cobertura.
- No hay pruebas de extremo a extremo ni de interfaz.
- No hay pruebas de los scripts de shell (por ejemplo con `bats` o `shellcheck`).

6. Integración continua

`.github/workflows/ci.yml`

Trabajo	Cuándo se ejecuta	Qué hace
<code>changes</code>	Siempre	<code>dorny/paths-filter</code> calcula tres banderas: <code>docs (**.md, docs/**)</code> , <code>src (src/**, src-tauri/**, package.json, pnpm-lock.yaml, tsconfig.json)</code> y <code>manifests (apps/**)</code>
<code>lint-docs</code>	Si <code>docs == true</code>	<code>markdownlint-cli2</code> sobre <code>**/*.md</code>
<code>typecheck</code>	Si <code>src == true</code>	<code>pnpm install --frozen-lockfile + tsc --noEmit</code>
<code>test-frontend</code>	Si <code>src == true</code>	<code>pnpm test</code>
<code>test-rust</code>	Si <code>src == true</code>	Instala dependencias GTK/WebKit y ejecuta <code>cargo test --no-default-features</code>
<code>validate-manifests</code>	Si <code>manifests == true</code>	Valida los YAML de <code>apps/</code> con PyYAML

Consecuencia importante del filtrado: **un cambio que sólo toca documentación ejecuta únicamente el lint de Markdown**. No se comprueban tipos ni se ejecutan pruebas, lo cual es correcto y rápido, pero conviene tenerlo presente al interpretar un CI en verde.

`.github/workflows/security.yml`

Se dispara en push y pull request a `main`, los lunes a las 06:00 UTC, manualmente y también por `workflow_call` desde otros repositorios. Incluye el escaneo de secretos, las auditorías de npm y cargo, CodeQL y la comprobación de acciones fijadas. El trabajo de `pnpm audit` falla si hay vulnerabilidades de severidad alta o crítica en dependencias de producción.

7. Datos y fixtures de prueba

No hay archivos de fixture. Todas las pruebas usan literales dentro del propio archivo: cadenas de log en `utils.test.ts` y claves de diccionario en `i18n.test.ts`. Las pruebas de Rust usan el proceso actual y el sistema de archivos real (`/`), lo que las hace dependientes del entorno: `read_disk_usage_returns_two_values` fallaría en un contenedor donde `df` no devolviera el formato esperado.

8. Casos límite cubiertos y no cubiertos

Cubiertos hoy:

- Líneas de log con códigos ANSI, porcentajes fuera de rango y líneas sin patrón conocido.

Tamaños de cero, nulos e indefinidos en el formateo de bytes.

- Claves de traducción ausentes e idiomas no soportados.
- PIDs vivos e inexistentes.

No cubiertos, y son los que más duelen:

- `resolve_effective_home` con volumen desmontado, `sparsebundle` presente o ausente, y `fallback`.
- `manifest_install_dir` con `override` absoluto, relativo y sin `override`.
- La guardia de recorrido de rutas de `delete_tool_model` **ejercitada de verdad**.
- `validate_workflow_id` con entradas maliciosas.
- `platform_supported` con manifiestos sin campo `platforms`.
- La post-validación de `installed_if` tras una instalación aparentemente exitosa.
- Cualquier comportamiento de la interfaz: cola, temporizadores, ventana de 60 segundos del health check, modo web sin backend.

9. Propuesta priorizada de pruebas faltantes

Ninguna de estas pruebas se ha implementado: este documento sólo las propone.

Prioridad	Módulo	Prueba propuesta	Por qué importa	Dificultad
ChofyAI Studio · 12 · Pruebas y calidad 1	system.rs	delete_tool_mode l con ../etc/passwd sobre un directorio temporal	Es el control de seguridad más importante y hoy está falsamente probado	v0.5.1 (commit f840055) · 2026-08-28 Media: requiere refactorizar la guardia a una función pura
2	system.rs	resolve_effective_home con las cuatro combinaciones de ruta usable, sparsebundle y fallback	Es el corazón del sistema; un fallo aquí deja al usuario sin herramientas	Media: requiere inyectar rutas en vez de leer settings
3	system.rs	manifest_install_dir con override absoluto, relativo y ausente	Función pura, barata de probar, alto impacto	Baja
4	system.rs	validate_workflow_id con .., /, cadena vacía y caracteres Unicode	Función pura ya aislada	Baja
5	system.rs	resolve_install_script y resolve_run_command por plataforma, incluida la retrocompatibilidad sin platforms	Regula qué se ejecuta en cada sistema	Baja
6	lib.rs	Prueba que compare la lista del invoke_handler con las funciones #[tauri::command] de system.rs	Evita comandos implementados y no registrados	Media
7	App.tsx	Extraer la lógica de la cola a un módulo puro y probarla	Hoy no hay forma de probar el flujo más visible del producto	Alta: exige refactor
8	apps/*.yaml	Ampliar el validador de CI para exigir que exista el script referenciado por install_scripts	Detectaría los scripts/linux/* inexistentes	Baja
9	scripts/mac/command.sh	Pruebas con bats de resolve_studio_home y _path_is_usable	Duplican lógica de Rust y pueden divergir	Media
10	Todo el repositorio	shellcheck en CI para los scripts de macOS	Detección estática barata	Baja

Prioridad	Módulo	Prueba propuesta	Por qué importa	Dificultad
ChofyAI Studio · 12 · Pruebas y 11	calidad workflows/*.yaml	Validación de esquema contra WorkflowDef en CI	Hoy un workflow malformado llega al render	v0.5.1 (commit f840055) · 2026-08-28 Baja
12	Frontend	Medición de cobertura con vitest --coverage	Permitiría hablar de números en vez de impresiones	Baja

10. Criterios de aceptación sugeridos

Alineados con lo que ya verifica CI, para que un cambio se considere listo:

- pnpm exec tsc --noEmit sin errores.
- pnpm test en verde.
- cargo test --no-default-features en verde.
- markdownlint sin avisos si se tocó documentación.
- El validador de manifiestos en verde si se tocó apps/ .
- Si el cambio añade un comando Tauri, debe estar registrado en lib.rs , tipado en types.ts y documentado en 05 · Referencia técnica.
- Si el cambio añade una clave de traducción, debe existir en los dos idiomas: la prueba de paridad de i18n.test.ts lo verifica automáticamente.